

🌱 1. What is Servlet?

A **Servlet** is a **Java program that runs on a server** and acts as a **middle layer** between:

- **Client (browser)** requests, and
- **Server-side resources (like databases or files).**

It is part of the **Java EE (Jakarta EE)** platform and is mainly used for building **dynamic web applications**.

🌐 2. Servlet Architecture Overview

When a client sends a request (like clicking a button or submitting a form):

1. **Browser (Client)** → sends an **HTTP request**
2. **Web Server (Tomcat, Jetty, etc.)** → forwards it to the **Servlet Container**
3. **Servlet Container** → executes the **Servlet class**
4. **Servlet** → generates a **response (HTML, JSON, etc.)**
5. **Response** → goes back to the **browser**

⚙️ 3. Servlet Life Cycle

A servlet's life is managed by the **Servlet Container** (like Apache Tomcat).

The **Servlet Life Cycle** consists of **5 phases** 🙌

Phase	Method	Description
1. Loading and Instantiation	<code>init()</code>	Servlet is loaded into memory and initialized (only once).
2. Initialization	<code>init(ServletConfig config)</code>	Called once when the servlet is first loaded; used to initialize resources.
3. Request Handling	<code>service(ServletRequest req, ServletResponse res)</code>	Called every time a request comes in.
4. Destruction	<code>destroy()</code>	Called once before the servlet is destroyed or unloaded.
5. Garbage Collection	—	JVM removes servlet object from memory.

4. Important Servlet Methods

Method	Description
<code>init()</code>	Initializes servlet when first loaded.
<code>service()</code>	Handles requests and responses.
<code>doGet()</code>	Handles HTTP GET requests (like typing a URL).
<code>doPost()</code>	Handles HTTP POST requests (like form submission).
<code>destroy()</code>	Cleans up resources before servlet is destroyed.



7. JSP (Java Server Pages)

JSP is used to write dynamic web pages easily using HTML + Java code.

It's built on top of Servlets.

Servlet	JSP
Java code inside HTML (logic first)	HTML with Java code inside (view first)
Harder to write UI	Easier for front-end pages
Best for backend logic	Best for presentation layer



8. JSP Life Cycle

JSP is internally converted into a Servlet by the server.

Phase	Description
1. Translation	JSP is translated into a servlet (.java file).
2. Compilation	Servlet is compiled into a .class file.
3. Initialization	<code>jspInit()</code> method is called once.
4. Execution	<code>_jspService()</code> method called for each request.
5. Destruction	<code>jspDestroy()</code> method called before unloading.



10. Servlet vs JSP (Summary Table)

Feature	Servlet	JSP
Syntax	Java code	HTML-like
Best For	Logic	Presentation
Compilation	Manual (in class)	Automatic (server converts JSP to servlet)
Extension	.java	.jsp
Easy for UI?	✗	✓

Advanced Java

Servlet :-

What is Servlet ?

- A Servlet is a Java program that runs on a web server.
- It is used to create dynamic web content.
- It acts as a middle layer between the client request and the server.
- It connects the browser with server-side resources like databases or applications.

Why Use Servlet ?

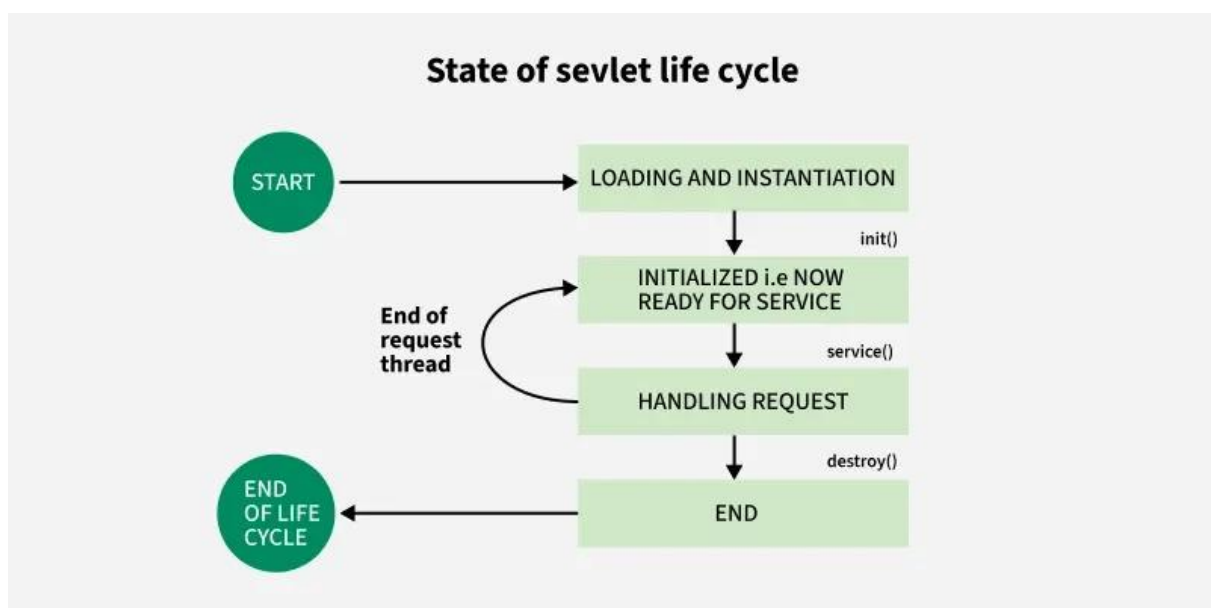
- To create dynamic web pages using Java.
- To handle client requests and server responses efficiently.
- To build secure, robust, and platform-independent web applications.
- To improve performance compared to traditional CGI (Common Gateway Interface).
- To easily connect with databases and other backend systems.
- To take advantage of reusability and scalability in web development.

How it Works ?

- The client (usually a web browser) sends a request to the web server.
- The web server passes the request to the Servlet container (like Tomcat).
- The Servlet container loads the required Servlet class.
- The Servlet processes the request (e.g., reads data, interacts with a database).
- The Servlet generates a response (usually HTML).
- The web server sends the response back to the client browser.

Life Cycle of Servlet :-

The Servlet life cycle consists of four main stages:



1. Loading and Instantiation

- The servlet class is loaded into memory by the container.
- Then, an instance of the servlet is created.

2. Initialization (`init()` method)

- The container calls the `init()` method once.
- It is used to initialize the servlet before handling requests.

3. Request Handling (`service()` method)

- The `service()` method is called for every client request.
- It processes the request and sends back a response.

4. Destruction (`destroy()` method)

- The `destroy()` method is called once before the servlet is unloaded.
- It is used to release resources or perform cleanup.

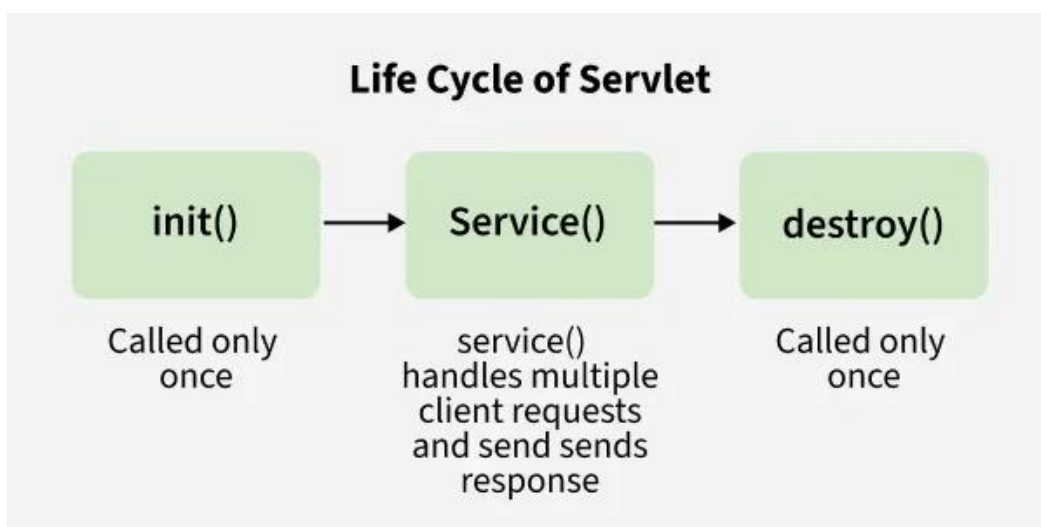
5. Garbage Collection

- After destruction, the servlet object is no longer needed.
- It becomes eligible for garbage collection by the JVM.

Servlet Life Cycle Methods :-

There are three life cycle methods of a Servlet:

1. `init()`
2. `service()`
3. `destroy()`



1. init() :-

- This method is called only once when the servlet is first loaded.
- It is used for initializing resources like database connections or configuration settings.

2. service() :-

- This method is called each time a client request is received.
- It is responsible for processing requests and generating responses.

3. destroy() :-

- This method is called once when the servlet is being removed from memory.
- It is used to release resources and perform cleanup tasks.

Java Server Page (JSP) :-

What is JSP ?

- JSP is a server-side technology used to create dynamic web content.
- It allows Java code to be embedded directly into HTML pages.
- JSP pages are executed on the server to generate dynamic responses for clients.
- It is an extension of the Servlet technology, making web development easier and faster.

Why Use JSP ?

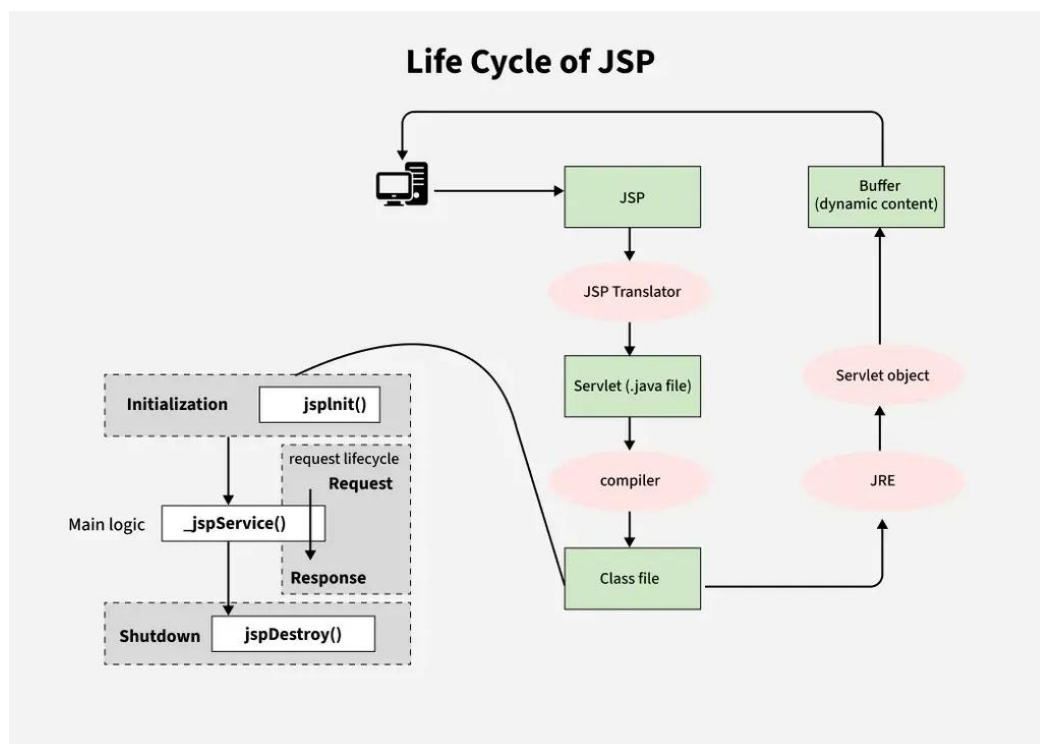
- To create dynamic web pages easily.
- Simpler and faster than servlets.
- Separates design from business logic.
- Supports reusable components.
- Automatically compiled and platform-independent.

How it Works ?

- The client sends a request for a .jsp page.
- The web server passes the request to the JSP engine.
- The JSP page is converted into a Servlet by the container.
- The Servlet is then compiled and executed to generate HTML output.
- The response is sent back to the client browser.

Life Cycle of JSP :-

The JSP life cycle consists of five main phases:



- The life cycle of JSP includes several phases from creation to destruction.
- The JSP page is first created and then translated into a servlet.
- The servlet lifecycle manages its execution and response handling.
- The entire process is handled automatically by the JSP engine.

1. Translation Phase

- The JSP file is translated into a Java Servlet by the JSP container.
- This happens only once when the page is first requested.

2. Compilation Phase

- The translated servlet file is compiled into bytecode.
- This bytecode is then loaded into memory for execution.

3. Initialization (jspInit() method)

- The container calls the jspInit() method once to initialize the JSP.
- It is used to set up resources needed for processing requests.

4. Request Processing (_jspService() method)

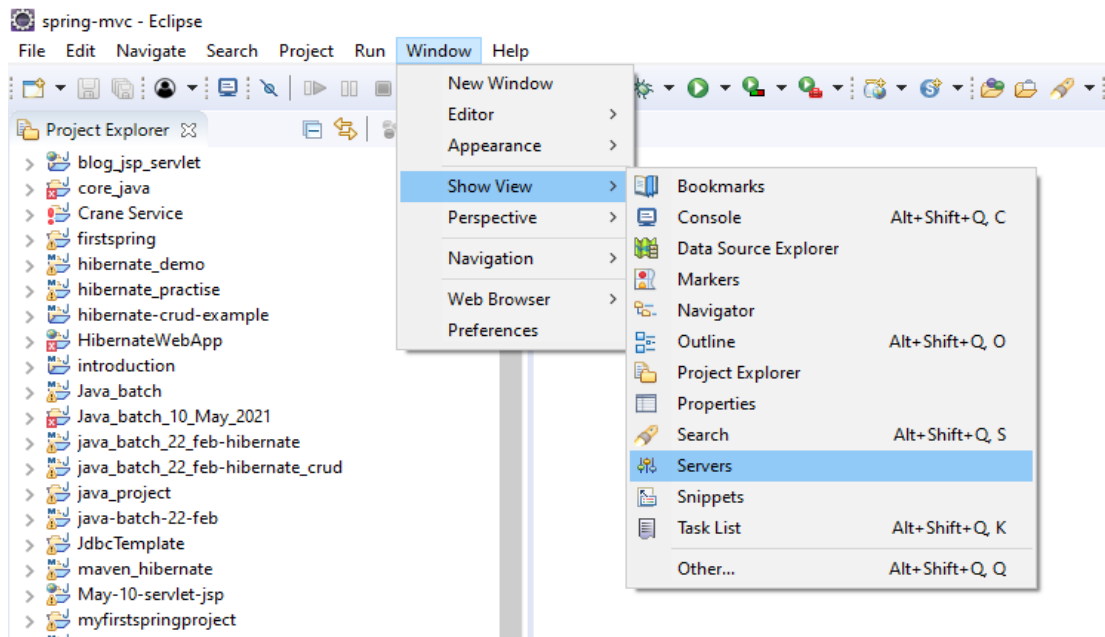
- The _jspService() method is called for each client request.
- It handles the request and generates a dynamic response.

5. Destruction (jspDestroy() method)

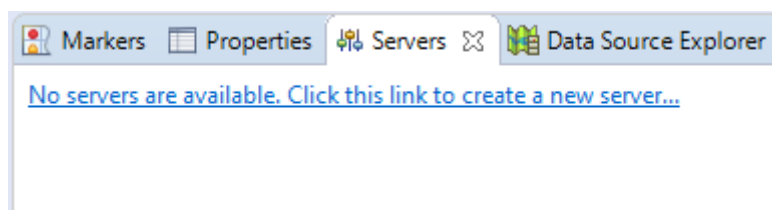
- The jspDestroy() method is called when the JSP is about to be removed.
- It is used to release resources and perform cleanup tasks.

How to add tomcat Server ?

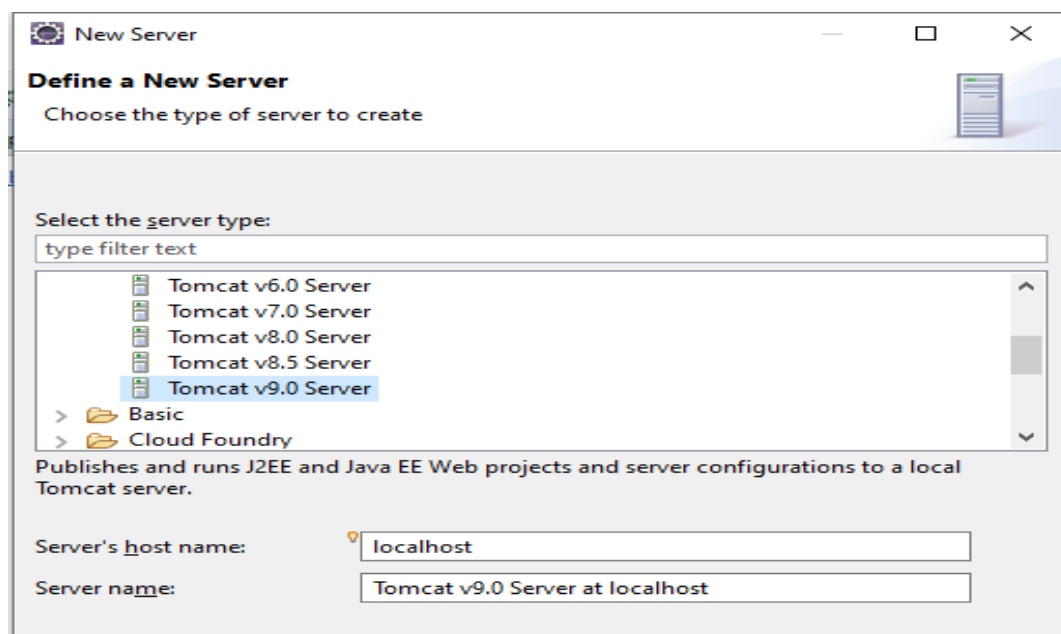
1. Step 1



2. Step 2

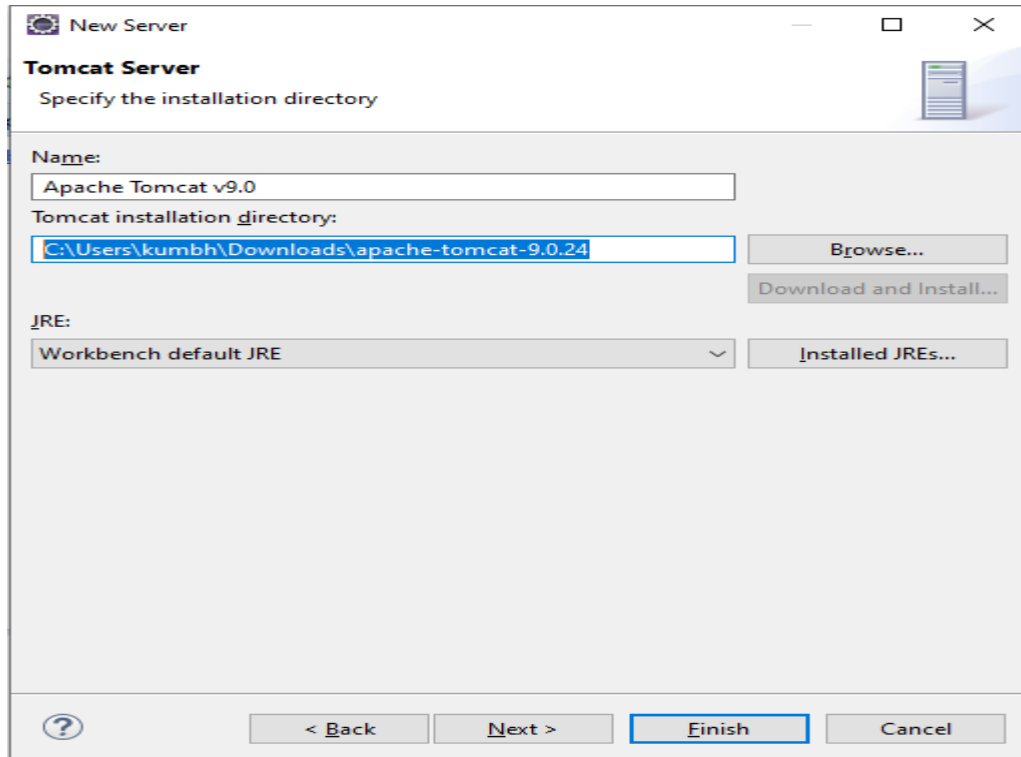


3. Open Apache folder- check & download your tomcat version zip file & extract it



4. Click Next

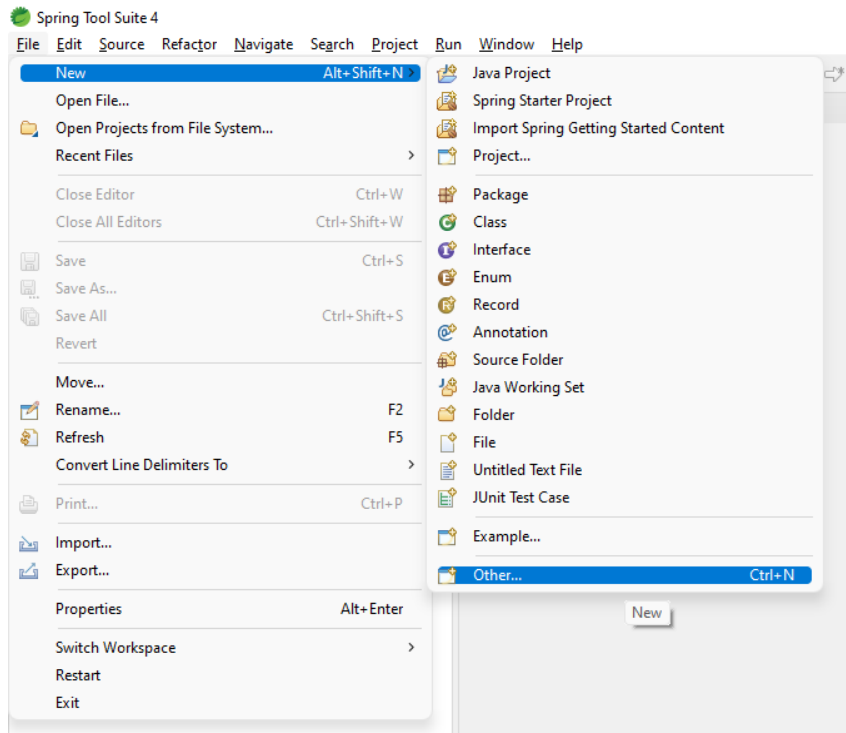
5. Browse tomcat folder from your folder



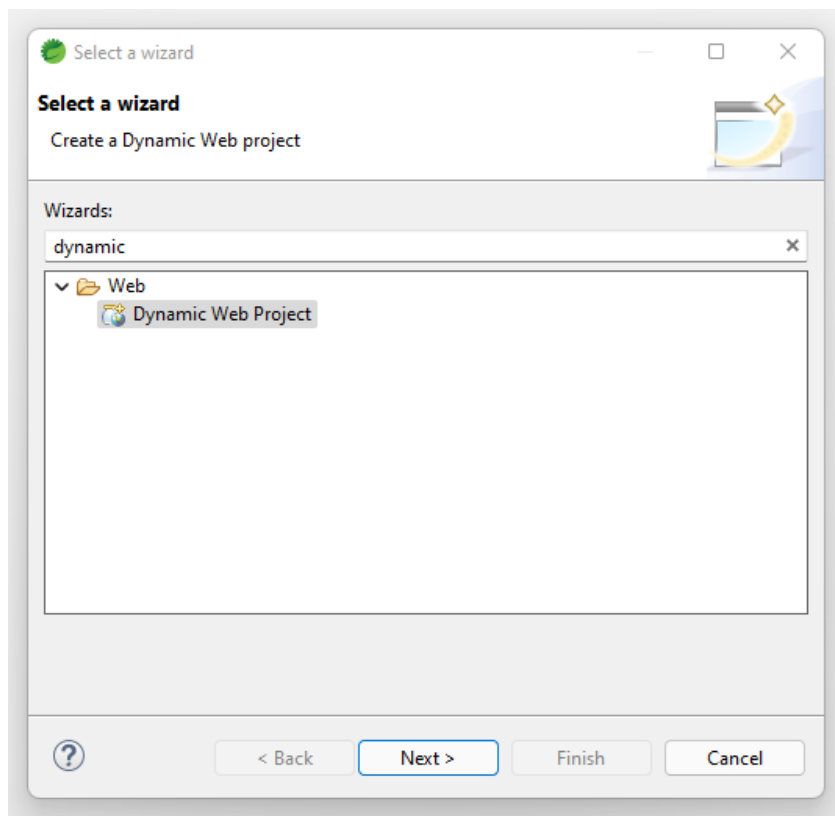
6. Finish

How to Create a Dynamic Web Project in Eclipse IDE ?

Step 1: Open your Eclipse IDE then go to the File > New > Other as shown in the below image.



Step 2: Now in the select wizard search for the Dynamic Web Project as shown in the below image. And click on the Next button.



Step 3: In the next screen you have to provide your project name as per your choice and don't touch anything else for now. Click on the Next button.

New Dynamic Web Project

Dynamic Web Project
Create a standalone Java-based Web Application or add it to a new or existing Enterprise Application.

Project name:

Project location

☒ Use default location

Location:

Target runtime

Dynamic web module version

Configuration

A good starting point for working with Apache Tomcat v9.0 runtime. Additional facets can later be installed to add new functionality to the project.

EAR membership

☐ Add project to an EAR

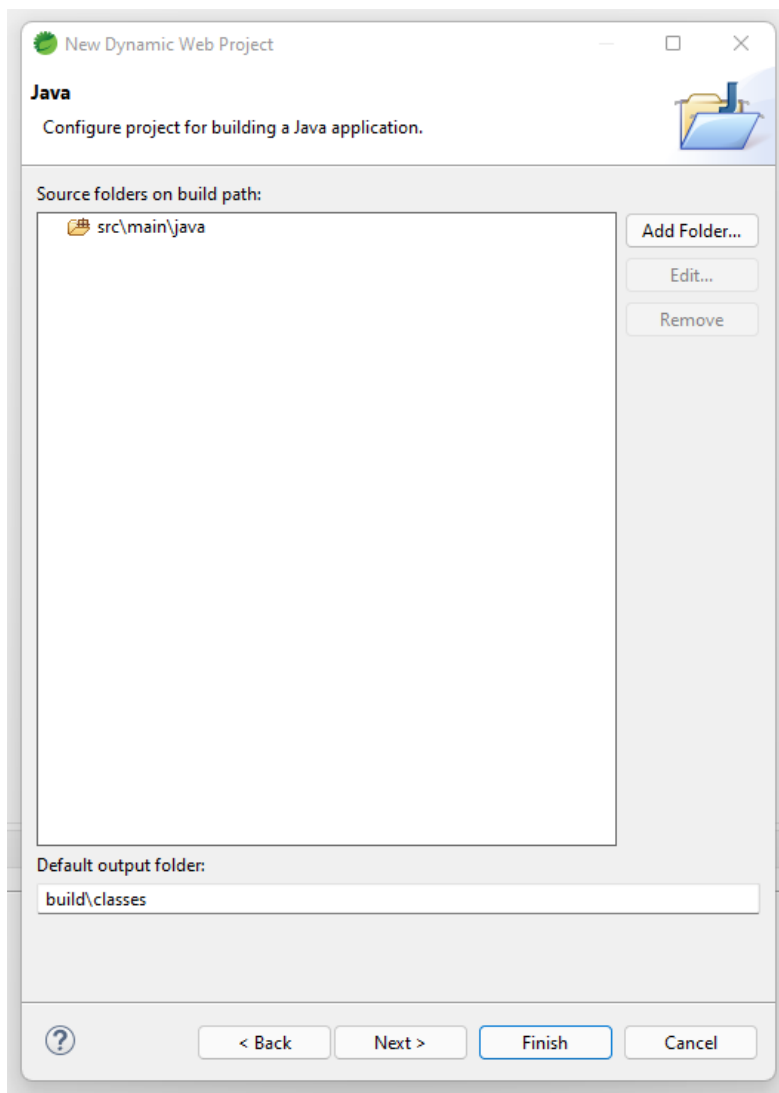
EAR project name:

Working sets

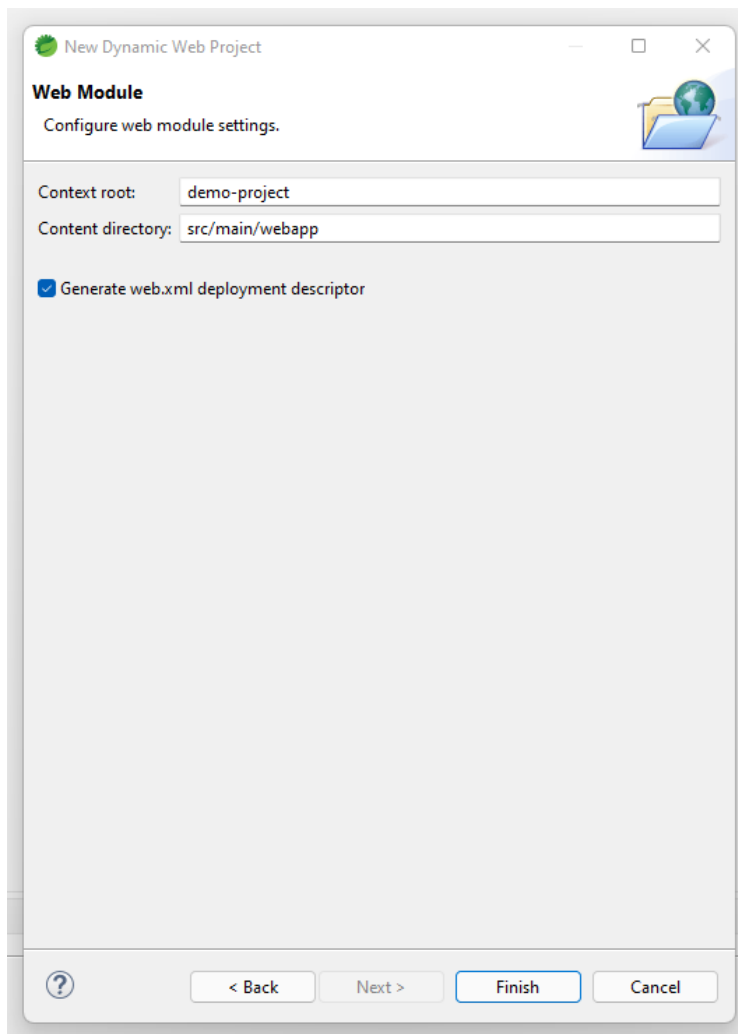
☐ Add project to working sets

Working sets:

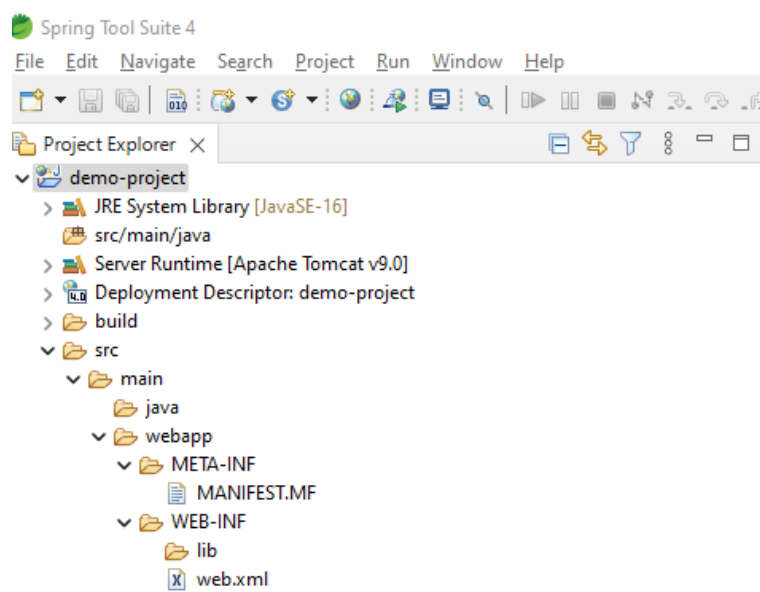
Step 4: In the next screen just click on the Next button.



Step 5: In the last screen just check out the Generate web.xml deployment descriptor box because we need it during the development of a Spring MVC project. Now click on the Finish button and you are done.

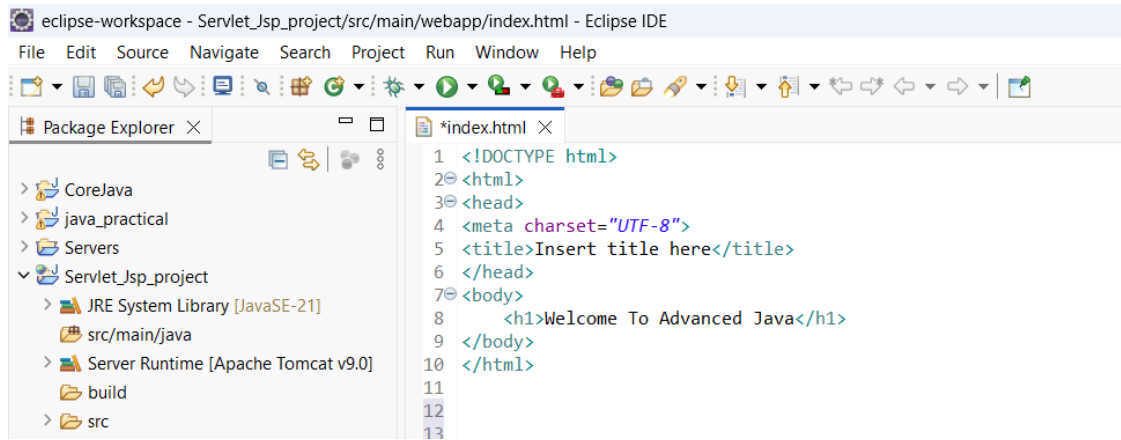


Now your Dynamic Web Project is ready and below is the file structure.



Implementation of JSP_Servlet :-

Example :-



Output :-



Servlet & JSP

Steps to Create a First Servlet :-

1. Create a Dynamic Web Project in Eclipse or any IDE.
2. Configure the Apache Tomcat server with your project.
3. Create a new Servlet class inside the src folder.
4. Extend the class from the HttpServlet class.
5. Override the doGet() or doPost() method.
6. Use the response.setContentType("text/html") method to set the output type.
7. Use PrintWriter out = response.getWriter(); to write output.
8. Print HTML output using out.println() statements.
9. Configure the servlet in the web.xml file using <servlet> and <servlet-mapping> tags.
10. Create an HTML file to send a request to the servlet using a form or link.

HTTP Request in Servlet :-

- HTTP (Hypertext Transfer Protocol) is used for communication between client and server.
- Each request uses a specific HTTP method for different operations.
- In servlets, each method is handled by a matching method in HttpServlet class.
- The servlet container automatically calls the correct method based on the client request.

Types of HTTP Request in Servlet :-

1. doGet Method :-

- **Description:** Handles HTTP GET requests.
- **Use Case:** Ideal for retrieving information from the server.

Parameters are appended to the URL.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handling GET requests
        PrintWriter out = response.getWriter();
        out.println("Hello, this is a GET request response!");
    }
}
```

2. doPost Method :-

- **Description:** Handles HTTP POST requests.
- **Use Case:** Used for submitting data to be processed to a specified resource. Commonly used for form submissions.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handling POST requests
        String data = request.getParameter("data"); // Retrieve data from the req
        // Process the data
        PrintWriter out = response.getWriter();
        out.println("Received data: " + data);
    }
}
```

3. doPut Method :-

- **Description:** Handles HTTP PUT requests.
- **Use Case:** Typically used for updating a resource on the server. The entire representation of the resource is sent in the request.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doPut(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handling PUT requests
        // Extract data from the request and update a resource
        PrintWriter out = response.getWriter();
        out.println("Resource updated successfully!");
    }
}
```

4. doDelete Method:

- **Description:** Handles HTTP DELETE requests.
- **Use Case:** Used to request that a resource be removed. Can delete a specified resource on the server.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doDelete(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        // Handling DELETE requests
        // Delete a resource based on the request
        PrintWriter out = response.getWriter();
        out.println("Resource deleted successfully!");
    }
}
```

Difference between doGet() and doPost() in Servlet :-

Basis	doGet()	doPost()
Purpose	Used to handle HTTP GET requests.	Used to handle HTTP POST requests.
Data Transmission	Data is sent through the URL as query parameters.	Data is sent in the request body.
Visibility	Data is visible in the browser's address bar.	Data is not visible in the URL.
Data Size Limit	Limited amount of data can be sent (URL length limit).	Large amounts of data can be sent.
Caching	GET requests can be cached by the browser.	POST requests are not cached.
Bookmarking	Can be bookmarked because parameters are in the URL.	Cannot be bookmarked.
Use Case	Used for retrieving data from the server.	Used for sending or submitting data to the server.
Idempotency	Generally idempotent (no change in server state).	Non-idempotent (may change server state).

Example :-

Addition.java

```
package Demo;

import java.io.IOException;

@WebServlet("/Addition")
public class Addition extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        int n1=Integer.parseInt(request.getParameter("num1"));
        int n2=Integer.parseInt(request.getParameter("num2"));

        int result=n1+n2;

        System.out.println("Addition= "+result);
    }
}
```

web.xml Configuration (if annotation not used) :-

```
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd (xsi:schemaLocation with catalog)
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">

    <servlet>
        <servlet-name>abc</servlet-name>
        <servlet-class>Demo.Addition</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>abc</servlet-name>
        <url-pattern>/add</url-pattern>
    </servlet-mapping>

</web-app>
```

HTML File (index.html) :-

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

    <h1>Welcome To Advanced Java</h1>

    <form action="add" method="post">
        <input type="number" placeholder="Enter number 1" name="num1"><br><br>
        <input type="number" placeholder="Enter number 2" name="num2"><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

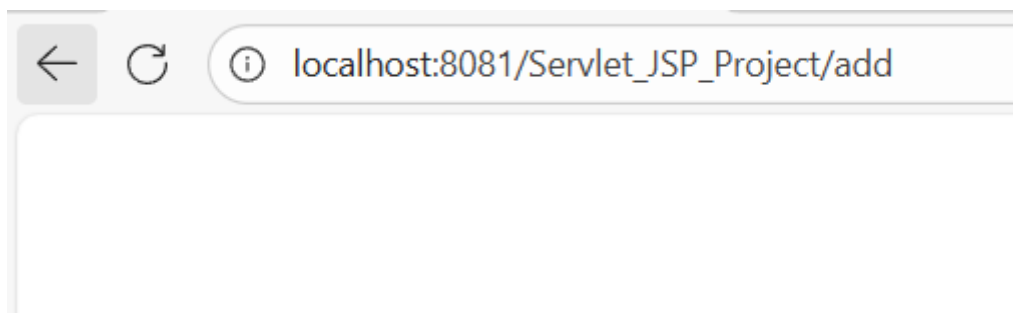
Output :-

HTML Page -



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/index.html'. The page content includes a heading 'Welcome To Advanced Java' in a large, bold, black serif font. Below the heading are two text input fields. The first field contains the number '5' and the second field contains the number '4'. Below these fields is a button labeled 'Submit'.

Web Page –



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/add'. The page content is empty.

Console Page -

Addition=9

Steps to Create a JSP Page in a Servlet Project :-

1. Create a Dynamic Web Project in Eclipse or any Java IDE.
2. Configure the Apache Tomcat server with your project.
3. Inside the WebContent folder, create a new JSP file (for example, index.jsp).
4. Write basic HTML structure in the JSP file.
5. Add JSP scripting elements if needed (<% %>, <%= %>, or <%! %>).
6. Create a Servlet class inside the src folder.
7. Extend the servlet class from HttpServlet.
8. In the servlet, process user input or perform backend logic.
9. Use request.setAttribute() to send data from servlet to JSP.
10. Forward the request from servlet to JSP using
RequestDispatcher rd = request.getRequestDispatcher("index.jsp");
11. Call rd.forward(request, response); to display the JSP page.
12. The JSP page receives data from the servlet and displays it in the browser.

Example: -

HTML File (index.html) :-

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

<h1>Welcome To Advanced Java</h1>

    <form action="addition.jsp" method="post">
        <input type="number" placeholder="Enter number 1" name="num1"><br><br>
        <input type="number" placeholder="Enter number 2" name="num2"><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

JSP File (addition.jsp) :-

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
        int n1=Integer.parseInt(request.getParameter("num1"));
        int n2=Integer.parseInt(request.getParameter("num2"));

        int result=n1+n2;

        out.println("Addition= "+result);
    %>
</body>
</html>
```

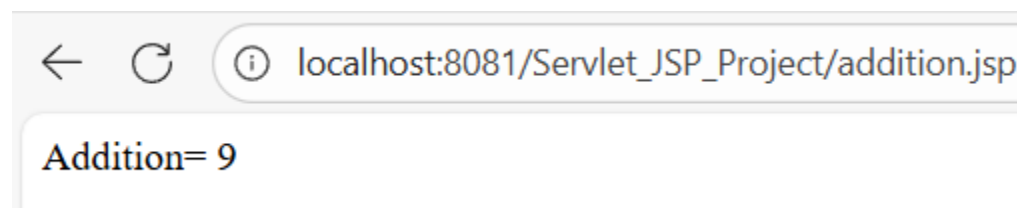
Output :-

HTML Page –



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/index.html'. The main content area displays 'Welcome To Advanced Java' in a large, bold, black serif font. Below the text are two input fields: the first contains the number '5' and the second contains the number '4'. Below these fields is a 'Submit' button.

JSP Page –



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/addition.jsp'. The main content area displays 'Addition= 9' in a black serif font.

Task – Registration Form using JDBC :-

JSP File (index.jsp) :-

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Registration Form</h1>

    <form action="registration.jsp" method="post">
        <input type="text" name="fullname" placeholder="Enter your fullname"><br><br>
        <input type="email" name="email" placeholder="Enter your email"><br><br>
        <input type="password" name="password" placeholder="Enter your password"><br><br>
        <input type="number" name="contact" placeholder="Enter your contact number"><br><br>

        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

JSP File (registration.jsp) :-

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
        String fn=request.getParameter("fullname");
        String em=request.getParameter("email");
        String pss=request.getParameter("password");
        String cn=request.getParameter("contact");

        Class.forName("org.postgresql.Driver");

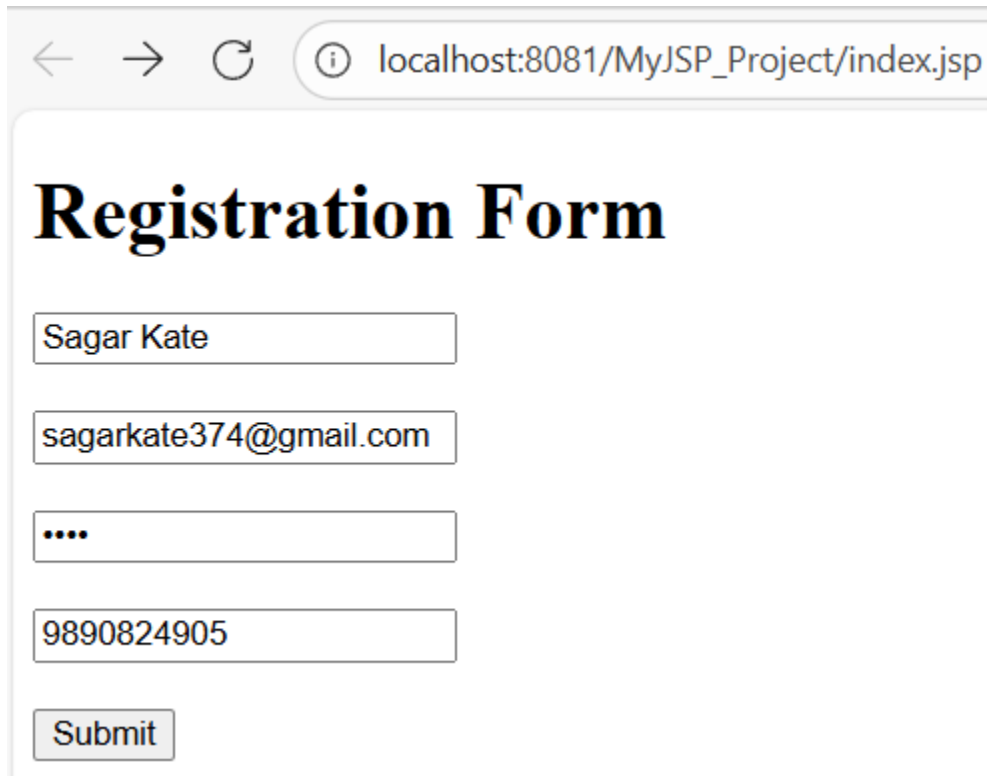
        String URL="jdbc:postgresql://localhost:5433/javabatch";
        String username="postgres";
        String password="4518";

        Connection con=DriverManager.getConnection(URL,username,password);

        String q = "insert into student(fullname,email,password,contact)values(?,?,?,?)";
        PreparedStatement ps = con.prepareStatement(q);
        ps.setString(1, fn);
        ps.setString(2, em);
        ps.setString(3, pss);
        ps.setString(4, cn);
        ps.executeUpdate();
        out.print("Registration Successfully Completed");
    %>
</body>
</html>
```

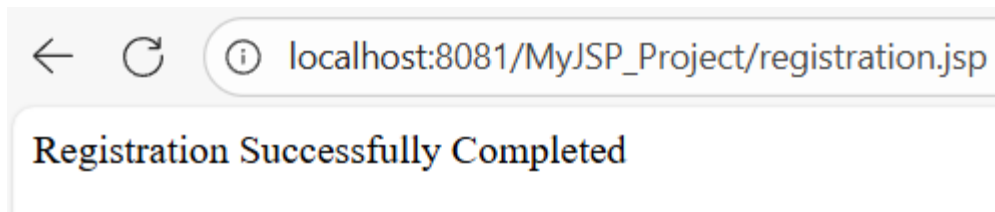
Output :-

Index.jsp Page –



The screenshot shows a web browser window with the address bar displaying 'localhost:8081/MyJSP_Project/index.jsp'. The page title is 'Registration Form'. The form contains five input fields: a text field for 'Sagar Kate', a text field for 'sagarkate374@gmail.com', a password field with four dots, a text field for '9890824905', and a 'Submit' button.

Registration.jsp Page –



The screenshot shows a web browser window with the address bar displaying 'localhost:8081/MyJSP_Project/registration.jsp'. The page title is 'Registration Successfully Completed'.

Database – student

	id integer 🔒	fullname character varying (100) 🔒	email character varying (100) 🔒	password character varying (100) 🔒	contact character varying (100) 🔒
1	[null]	Sagar Kate	sagarkate374@gmail.c...	5896	9890824905

Servlet & JSP

How to Perform JSP CRUD Application Using PostgreSQL ?

1. Create a PostgreSQL database and a table to store user data.
2. Design a JSP form to collect user input (name, email, etc.).
3. Use JDBC to connect JSP pages with the PostgreSQL database.
4. Write SQL queries to insert, display, update, and delete data.
5. Use PreparedStatement for executing queries securely.
6. Retrieve data using ResultSet and display it in a JSP table.
7. Redirect pages after each operation using response.sendRedirect().
8. Test all CRUD operations to ensure smooth database interaction.

How JSP Interacts with Databases Dynamically ?

1. **Database Connectivity:**
JSP uses **JDBC (Java Database Connectivity)** to establish a connection between the web application and the database.
2. **Driver Loading:**
The database driver (e.g., org.postgresql.Driver) is loaded using Class.forName() to enable communication with the specific database.
3. **Establishing Connection:**
A connection is created using DriverManager.getConnection(), which connects JSP to the database using the URL, username, and password.
4. **Executing SQL Queries:**
SQL queries like INSERT, SELECT, UPDATE, or DELETE are executed using **Statement** or **PreparedStatement** objects.
5. **Dynamic Data Handling:**
The JSP page retrieves user inputs with request.getParameter() and uses them in SQL queries to store or fetch data dynamically.
6. **Displaying Results:**
Data fetched from the database using a **ResultSet** is displayed on the web page in real-time using JSP expressions (<%= %>).
7. **Closing Connections:**
After execution, all database objects (ResultSet, Statement, Connection) are closed to release resources and maintain performance.

Modules of the Application :-

1. Registration Form (index.jsp)

- This page contains a simple **HTML form** that allows the user to input their details such as:
 - Full Name
 - Email
 - Password
 - Contact Number
- The form uses the POST method and submits the entered data to registration.jsp for processing.

2. Data Insertion (registration.jsp)

- This file retrieves the form data using the request.getParameter() method.
- It establishes a connection to the PostgreSQL database using the JDBC driver.
- A **PreparedStatement** is used to execute an INSERT query, which adds the user's data into the **student** table.
- Once the record is successfully inserted, the user is redirected to the **Welcome Page (welcome.jsp)** using response.sendRedirect().

Key Concepts Used:

- Class.forName("org.postgresql.Driver") – Loads PostgreSQL driver.
- DriverManager.getConnection() – Connects JSP to the PostgreSQL database.
- PreparedStatement – Used to safely insert data and prevent SQL Injection.

3. Display Records (welcome.jsp)

- This page retrieves all the stored records from the database using a SELECT * FROM student query.
- A **ResultSet** object is used to display data dynamically inside an HTML table.
- Each record is shown with a “Delete” link that passes the record ID to delete.jsp for deletion.

Purpose:

To view all registered users in a structured tabular format directly from the database.

4. Delete Operation (delete.jsp)

- This page performs the **DELETE** operation using the ID value passed from the welcome page.
- It prepares a SQL statement `DELETE FROM student WHERE id=?` to remove the selected record.
- After deleting, the page redirects back to the welcome page to refresh the record list.

Key Concept Used:

PreparedStatement ensures secure deletion of records without SQL injection risks.

Example – Registration Form

index.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Registration Form</h1>

    <form action="registration.jsp" method="post">
        <input type="text" name="fullname" placeholder="Enter your fullname"><br><br>
        <input type="email" name="email" placeholder="Enter your email"><br><br>
        <input type="password" name="password" placeholder="Enter your password"><br><br>
        <input type="number" name="contact" placeholder="Enter your contact number"><br><br>

        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

registration.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
        String fn=request.getParameter("fullname");
        String em=request.getParameter("email");
        String pss=request.getParameter("password");
        String cn=request.getParameter("contact");

        Class.forName("org.postgresql.Driver");

        String URL="jdbc:postgresql://localhost:5433/javabatch";
        String username="postgres";
        String password="4518";

        Connection con=DriverManager.getConnection(URL,username,password);

        String q = "insert into student(fullname,email,password,contact)values(?,?,?,?)";
```

```

        PreparedStatement ps = con.prepareStatement(q);
        ps.setString(1, fn);
        ps.setString(2, em);
        ps.setString(3, pss);
        ps.setString(4, cn);
        ps.executeUpdate();
        out.print("Registration Successfully Completed");

        response.sendRedirect("welcome.jsp");
    %>
</body>
</html>

```

welcome.jsp –

```

<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Welcome Page</h1>

    <table border="1">
    <tr>
        <th>Id</th>
        <th>Fullname</th>
        <th>Email Id</th>
        <th>Password</th>
        <th>Contact</th>
        <th></th>
    </tr>

    <%
        Class.forName("org.postgresql.Driver");

        String URL="jdbc:postgresql://localhost:5433/javabatch";
        String username="postgres";
        String password="4518";

        Connection con=DriverManager.getConnection(URL,username,password);

        Statement stmt=con.createStatement();
        ResultSet rs=stmt.executeQuery("select * from student");
        while(rs.next()){
    %>
    <tr>
        <td><%= rs.getInt(1) %></td>
        <td><%= rs.getString(2) %></td>
        <td><%= rs.getString(3) %></td>
        <td><%= rs.getString(4) %></td>
        <td><%= rs.getString(5) %></td>
        <td>
            <a href="delete.jsp?id=<%= rs.getInt(1) %>" onclick="return confirm('Are you sure ?')">Delete</a>
        </td>
    </tr>
    <% } %>
    </table>
</body>
</html>

```

delete.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
        int id=Integer.parseInt(request.getParameter("id"));

        Class.forName("org.postgresql.Driver");

        String URL="jdbc:postgresql://localhost:5433/javabatch";
        String username="postgres";
        String password="4518";

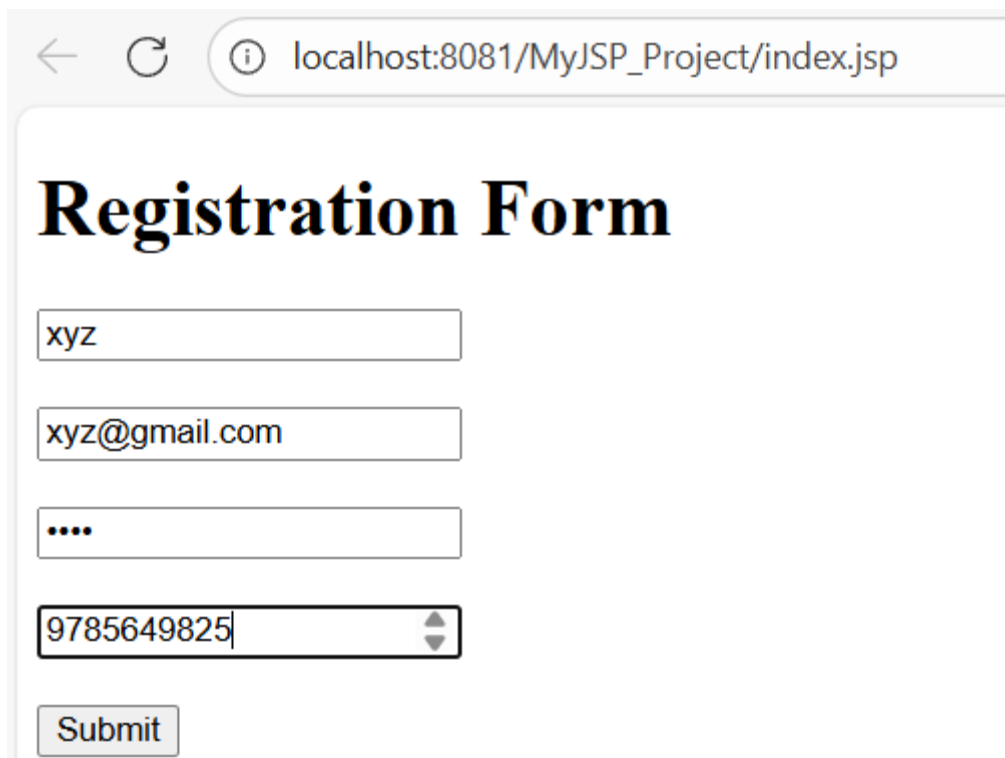
        Connection con=DriverManager.getConnection(URL,username,password);

        String q = "delete from student where id=?";
        PreparedStatement ps = con.prepareStatement(q);
        ps.setInt(1,id);
        ps.executeUpdate();

        response.sendRedirect("welcome.jsp");
    %>
</body>
</html>
```

Output –

Index.jsp Page -



The screenshot shows a web browser window with the address bar displaying "localhost:8081/MyJSP_Project/index.jsp". The page title is "Registration Form". The form contains five input fields: a text field with "xyz", a text field with "xyz@gmail.com", a text field with "....", a text field with "9785649825", and a "Submit" button.

localhost:8081/MyJSP_Project/index.jsp

Registration Form

xyz

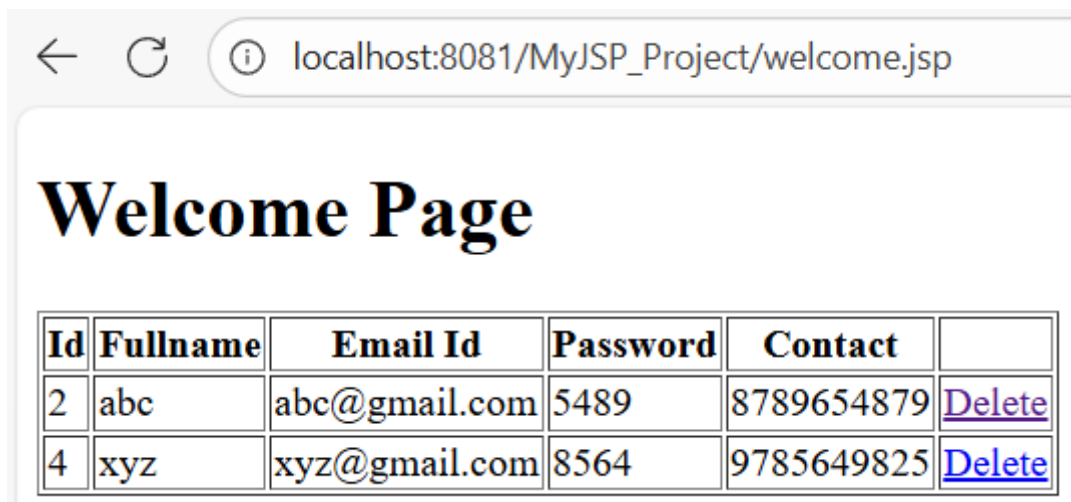
xyz@gmail.com

....

9785649825

Submit

welcome.jsp Page -

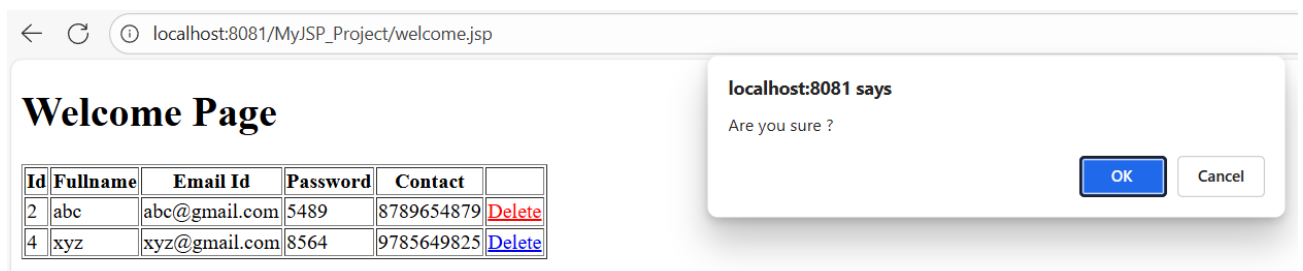


The screenshot shows a web browser window with the address bar displaying "localhost:8081/MyJSP_Project/welcome.jsp". The page title is "Welcome Page". Below the title is a table with five columns: Id, Fullname, Email Id, Password, and Contact. The table contains two rows of data. Each row has a "Delete" link in the last column.

localhost:8081/MyJSP_Project/welcome.jsp

Welcome Page

Id	Fullname	Email Id	Password	Contact	
2	abc	abc@gmail.com	5489	8789654879	Delete
4	xyz	xyz@gmail.com	8564	9785649825	Delete



The screenshot shows the same "Welcome Page" as above, but with a confirmation dialog box open. The dialog box has the title "localhost:8081 says" and the message "Are you sure ?". It contains two buttons: "OK" and "Cancel".

localhost:8081/MyJSP_Project/welcome.jsp

Welcome Page

Id	Fullname	Email Id	Password	Contact	
2	abc	abc@gmail.com	5489	8789654879	Delete
4	xyz	xyz@gmail.com	8564	9785649825	Delete

localhost:8081 says

Are you sure ?

OK Cancel

← ↻ ⓘ localhost:8081/MyJSP_Project/welcome.jsp

Welcome Page

Id	Fullname	Email Id	Password	Contact	
4	xyz	xyz@gmail.com	8564	9785649825	Delete

Database - Student

	id [PK] integer	fullname character varying (100)	email character varying (100)	password character varying (100)	contact character varying (15)
1	4	xyz	xyz@gmail.com	8564	9785649825

Servlet & JSP

Modules of the Application :-

1. Display Records (welcome.jsp)

- This page displays all student records from the database in a tabular format.
- It loads the PostgreSQL driver using `Class.forName("org.postgresql.Driver")` and connects to the database using `DriverManager.getConnection()`.
- A `Statement` object executes a `SELECT` query to fetch all records from the student table.
- The results are retrieved using a `ResultSet` and displayed dynamically inside an HTML `<table>` using JSP scriptlets.
- Each record includes two action links — Delete and Profile Edit for managing data.

Key Concepts Used:

1. `Class.forName("org.postgresql.Driver")` – Loads the PostgreSQL JDBC driver.
2. `DriverManager.getConnection()` – Establishes the database connection.
3. `Statement` and `ResultSet` – Used to fetch and display records.
4. `response.sendRedirect()` – Used later for page navigation.
5. JSP Scriptlets – For dynamically embedding Java code inside HTML.

2. Edit Records (profile_edit.jsp)

- This page retrieves a specific student record using the student's ID passed through the request parameter.
- It connects to the PostgreSQL database using JDBC to execute a `SELECT` query for that ID.
- The fetched data is displayed inside an HTML form with fields pre-filled using JSP expressions.
- The user can modify any detail and submit the form to save changes.
- The form sends the updated data to `profilesave.jsp` using the POST method.

Key Concepts Used:

1. `request.getParameter()` – Retrieves the student's ID from the URL.
2. `Class.forName()` and `DriverManager.getConnection()` – Establish database connection.
3. `ResultSet` – Fetches data for the selected student.
4. JSP Expression Tags `<%= %>` – Display dynamic values inside form fields.
5. Form Handling – Enables data update through POST submission.

3. Update Records (profilesave.jsp)

- This page receives updated data from the Edit Form using `request.getParameter()`.
- It loads the PostgreSQL driver and connects to the database using JDBC.
- A `PreparedStatement` executes an `UPDATE` query to modify existing data in the student table.
- The `PreparedStatement` ensures safe query execution and prevents SQL Injection.
- After the update, the user is redirected to `welcome.jsp` using `response.sendRedirect()`.

Key Concepts Used:

1. `PreparedStatement` – Safely updates records in the database.
2. `setString()` and `setInt()` – Used to set dynamic query parameters.
3. `executeUpdate()` – Executes the `UPDATE` query.
4. `response.sendRedirect()` – Redirects to the Welcome Page after updating.
5. JDBC + JSP Integration – Demonstrates server-side data update using JSP and PostgreSQL.

Example :-

welcome.jsp -

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Welcome Page</h1>

    <table border="1">
    <tr>
        <th>Id</th>
        <th>Fullname</th>
        <th>Email Id</th>
        <th>Password</th>
        <th>Contact</th>
        <th></th>
    </tr>

    <%
    Class.forName("org.postgresql.Driver");

    String URL="jdbc:postgresql://localhost:5433/javabatch";
    String username="postgres";
    String password="4518";

    Connection con=DriverManager.getConnection(URL,username,password);

    Statement stmt=con.createStatement();
    ResultSet rs=stmt.executeQuery("select * from student");
    while(rs.next()){
    %>

    <tr>
        <td><%= rs.getInt(1) %></td>
        <td><%= rs.getString(2) %></td>
        <td><%= rs.getString(3) %></td>
        <td><%= rs.getString(4) %></td>
        <td><%= rs.getString(5) %></td>
        <td>
            <a href="delete.jsp?id=<%= rs.getInt(1) %>" onclick="return confirm('Are you sure ?')">Delete</a>
            <a href="profile_edit.jsp?id=<%= rs.getInt(1) %>">Profile Edit</a>
        </td>
    </tr>
    <% } %>
    </table>
</body>
</html>
```

profile_edit.jsp -

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
```

```

        int id=Integer.parseInt(request.getParameter("id"));

        Class.forName("org.postgresql.Driver");

        String URL="jdbc:postgresql://localhost:5433/javabatch";
        String username="postgres";
        String password="4518";

        Connection con=DriverManager.getConnection(URL,username,password);

        Statement stmt=con.createStatement();
        ResultSet rs=stmt.executeQuery("select * from student where id="+id);
        while(rs.next()){

%>
<h1>Profile Edit Form</h1>

<form action="profilesave.jsp" method="post">
    <input type="hidden" name="id" value="<%= rs.getInt(1) %>"><br><br>
    <input type="text" name="fullname" placeholder="Enter your fullname" value="<%= rs.getString(2) %>"><br><br>
    <input type="email" name="email" placeholder="Enter your email" value="<%= rs.getString(3) %>"><br><br>
    <input type="password" name="password" placeholder="Enter your password" value="<%= rs.getString(4) %>"><br><br>
    <input type="number" name="contact" placeholder="Enter your contact number" value="<%= rs.getString(5) %>"><br><br>

    <button type="submit">Submit</button>
</form>

<% } %>

</body>
</html>

```

profilesave.jsp -

```

<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<%
    int id=Integer.parseInt(request.getParameter("id"));
    String fn=request.getParameter("fullname");
    String em=request.getParameter("email");
    String pss=request.getParameter("password");
    String cn=request.getParameter("contact");

    Class.forName("org.postgresql.Driver");

    String URL="jdbc:postgresql://localhost:5433/javabatch";
    String username="postgres";
    String password="4518";

    Connection con=DriverManager.getConnection(URL,username,password);

    String q = "update student set fullname=?, email=?, password=?, contact=? where id=?";
    PreparedStatement ps = con.prepareStatement(q);
    ps.setString(1, fn);
    ps.setString(2, em);
    ps.setString(3, pss);
    ps.setString(4, cn);

```

```
    ps.setInt(5, id);  
    ps.executeUpdate();  
    response.sendRedirect("welcome.jsp");  
%>  
</body>  
</html>
```

Output :-

Welcome Page -

← ↻ ⓘ localhost:8081/MyJSP_Project/welcome.jsp

Welcome Page

ID	Fullname	Email Id	Password	Contact	
4	xyz	xyz@gmail.com	8564	9785649825	Delete Profile Edit
5	abc	abc@gmail.com	6985	8975216945	Delete Profile Edit

Profile Edit Page -

← ↻ ⓘ localhost:8081/MyJSP_Project/profile_edit.jsp?id=4

Profile Edit Form

Profile Save Page -

← ↻ ⓘ localhost:8081/MyJSP_Project/welcome.jsp

Welcome Page

ID	Fullname	Email Id	Password	Contact	
5	abc	abc@gmail.com	6985	8975216945	Delete Profile Edit
4	xyz1	xyz1@gmail.com	85641	97856498251	Delete Profile Edit

What Is a Session in JSP?

- A session stores user data across multiple HTTP requests.
- HTTP is stateless, so it cannot remember users between requests.
- Sessions solve this by assigning each user a unique session ID for identification.

Each session is associated with one user and is maintained until:

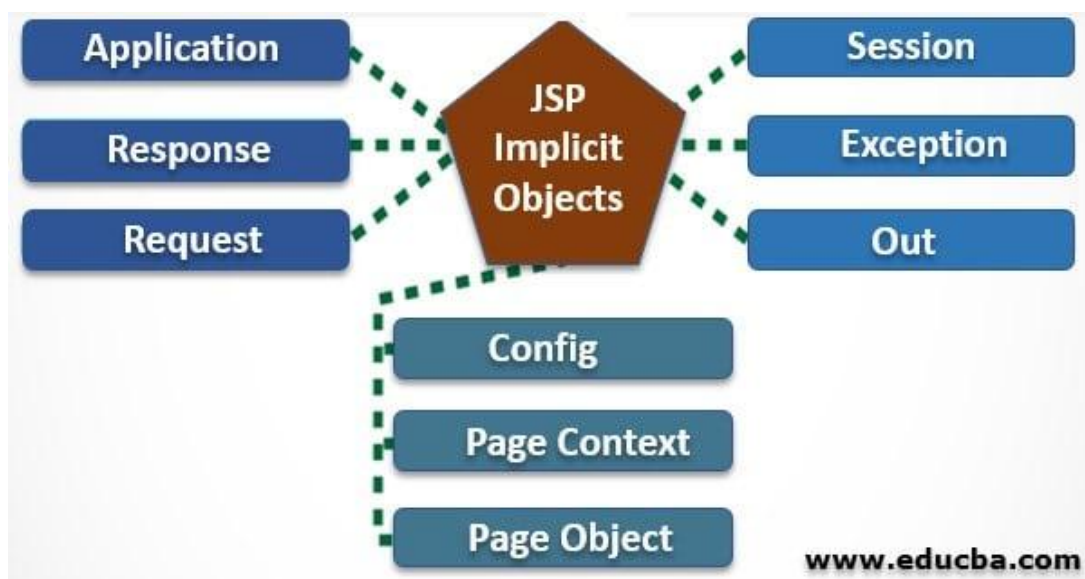
- The user logs out, or
- The session times out (default: 30 minutes), or
- The browser is closed.

How JSP Manages Session ?

- JSP manages sessions using the HttpSession object.
- Each user gets a unique session ID, usually stored in a cookie.
- The browser sends this ID with each request to identify the user.
- Data is stored using methods like `setAttribute()` and `getAttribute()`.
- The session lasts until it is invalidated or times out.

Session Object in JSP :-

- The session object in JSP is an implicit object used to maintain user data.
- It represents the HttpSession created for each user.
- Data can be stored and retrieved using `setAttribute()` and `getAttribute()`.
- Each session has a unique ID and lasts until it expires or is invalidated.



Common Session Methods :-

Method	Description
setAttribute(String name, Object value)	Stores a value in the session.
getAttribute(String name)	Retrieves a stored value.
removeAttribute(String name)	Deletes a session attribute.
invalidate()	Ends the session completely.
getId()	Returns the unique session ID.
getCreationTime()	Returns time when session was created.
getLastAccessedTime()	Returns last time the session was accessed.
isNew()	Returns true if the session is new.

Example :- Login System using JSP Session

login.jsp -

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

    <h1>Login Form</h1>

    <form action="logincheck.jsp" method="post">
        <input type="email" name="email" placeholder="Enter your email"><br><br>
        <input type="password" name="password" placeholder="Enter your password"><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

logincheck.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%

        String em=request.getParameter("email");
        String pss=request.getParameter("password");

        Class.forName("org.postgresql.Driver");

        String URL="jdbc:postgresql://localhost:5433/javabatch";
        String username="postgres";
        String password="4518";

        Connection con=DriverManager.getConnection(URL,username,password);

        Statement stmt=con.createStatement();
        ResultSet rs=stmt.executeQuery("select * from student where email='"+em+"' and password='"+pss+"'");
        if(rs.next()){

            HttpSession s1=request.getSession();
            s1.setAttribute("username",em);    //session start

            out.println("Login Successfully");
            response.sendRedirect("dashboard.jsp");
```

```

    }else{
        out.println("Login Fail");
        response.sendRedirect("login.jsp");
    }

%>

</body>
</html>

```

dashboard.jsp -

```

<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

    <%
        HttpSession s1=request.getSession(false);
        out.println(s1.getAttribute("username"));

        if(s1.getAttribute("username")==null){
            response.sendRedirect("login.jsp");
        }
    %>

    <h1>Welcome To Dashboard</h1>

    <br>
    <a href="logout.jsp">Click Here To Logout</a>

</body>
</html>

```

logout.jsp -

```

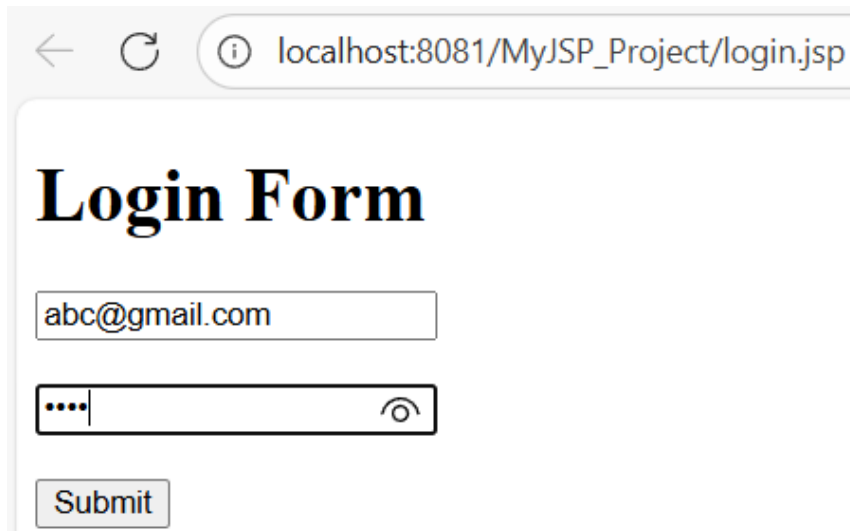
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
        HttpSession s1=request.getSession(false);
        response.sendRedirect("login.jsp");
    %>

</body>
</html>

```

Output :-

Login Page -

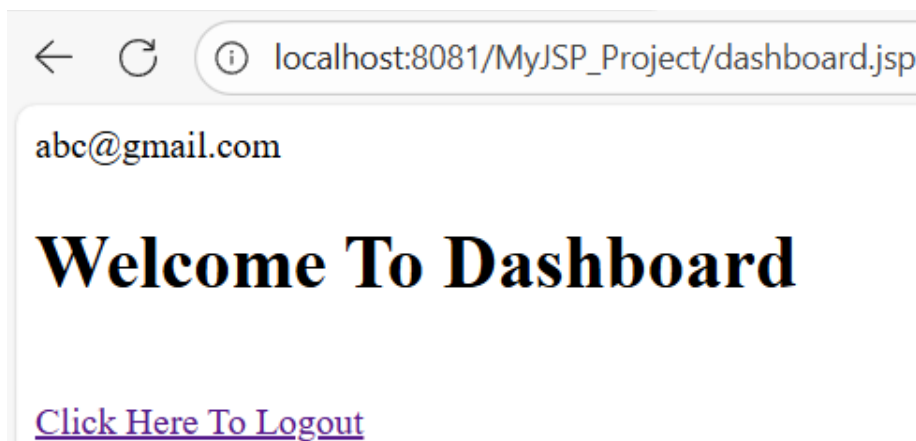


← ↻ ⓘ localhost:8081/MyJSP_Project/login.jsp

Login Form

🔍

Dashboard Page –



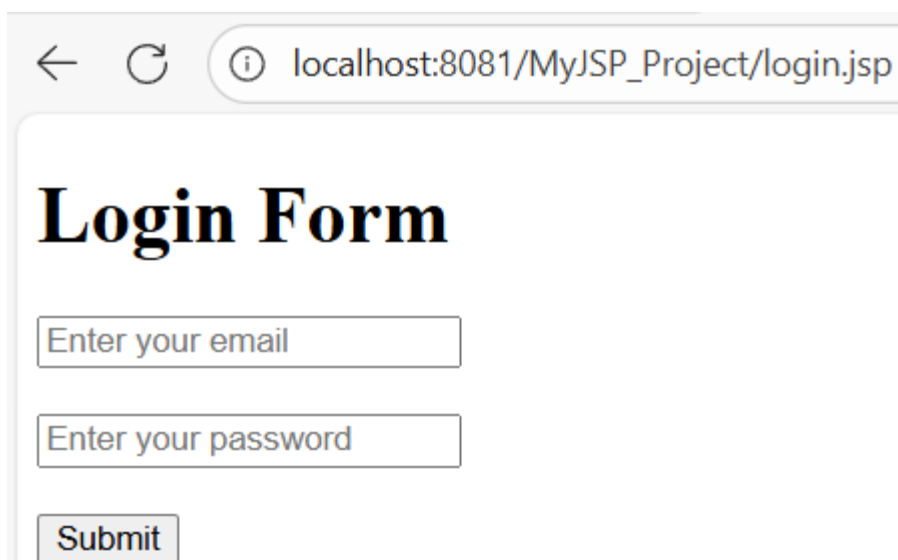
← ↻ ⓘ localhost:8081/MyJSP_Project/dashboard.jsp

abc@gmail.com

Welcome To Dashboard

[Click Here To Logout](#)

Logout Page –



← ↻ ⓘ localhost:8081/MyJSP_Project/login.jsp

Login Form